simulation engine used extensively by researchers, academics, and industry engineers. In IoT research, procuring and deploying hundreds of physical microcontroller boards, gateways, and radio transceivers is costly and logistically cumbersome.

ns-3 solves this by providing:

- Realistic simulation of network protocol stacks (IPv4, IPv6, UDP, TCP, point-to-point, 802.11, etc).
- Deterministic, repeatable experimentation to test network traffic, packet latency, routing behaviours, and protocol interactions under reproducible conditions.
- A flexible testbed for evaluating security mechanisms and simulating attacks (such as IP spoofing, unauthorised access, and man-in-the-middle exploits) in controlled environments without taking real operational infrastructure at risk.

The Zero Trust module for IoT (zero-trust-iot) extends ns-3 to enable researchers to model and observe how Zero Trust verification mechanisms function over standard network topologies.

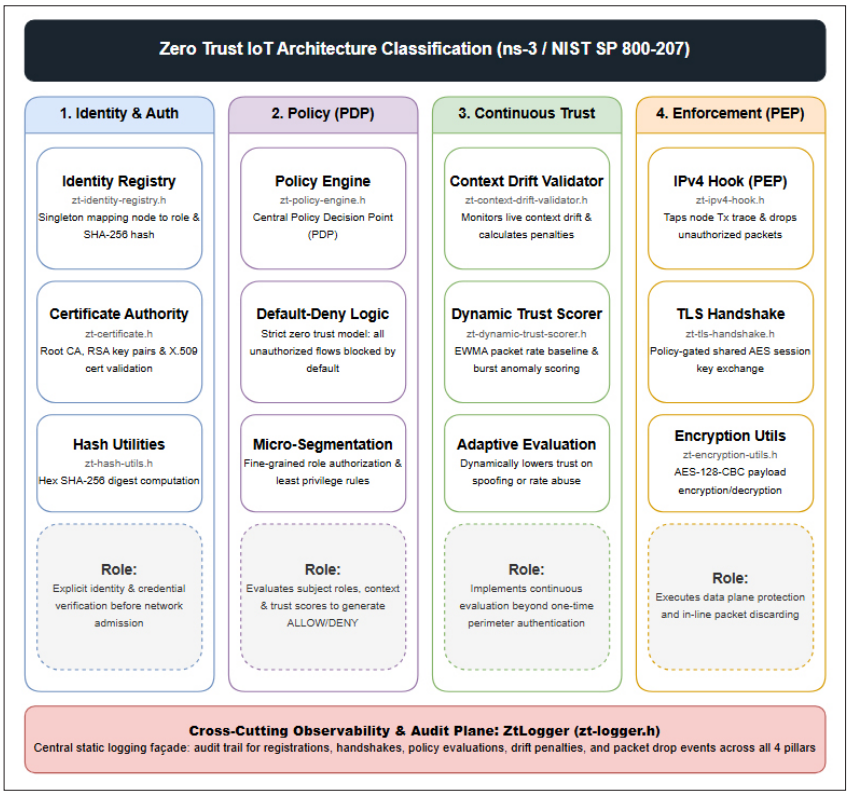


Figure 1: Zero Trust library architecture

Setup needed to simulate a Zero Trust module for IoT

To simulate Zero Trust IoT architectures, the module integrates directly with the ns-3 mainline source tree and relies on the Crypto++ (cryptopp) cryptographic library for cryptographic primitives (such as RSA certificate validation and AES encryption).

- ***Simulator:*** ns-3 (Tested and verified with releases ns-3.45, ns-3.46, and ns-3.47)
- ***Crypto library:*** Crypto++ (cryptopp)
- ***Build system:*** CMake and ns3 wrapper (./ns3)

Modules of the library

The library comprises ten coordinated C++ modules that work together to establish an enforceable trust boundary across every simulated IoT transaction as shown in Figure 1.

Identity registry (*IdentityRegistry*): This acts as a process-wide singleton that registers each *ns3::Node* alongside its assigned role string and a SHA-256 identity hash derived through *ComputeSha256*, enabling subsequent authorisation queries via *GetRole* and *GetIdentityHash*.

Hash utilities *(ComputeSha256):* Provides a standalone cryptographic utility wrapping Crypto++ SHA-256 to compute lowercase hexadecimal digests used for identity hashing, policy rule fingerprinting, and rule integrity protection.

Certificate authority and certificate-based policy (*CertificateAuthority, ZtPolicyEngineWithCert):* This module simulates lightweight public key infrastructure by generating 1024-bit RSA key pairs, issuing signed identity certificates with node identifier, role, and expiration timestamps via *SignIdentity*, and validating those credentials using RSA-PSS and an explicit revocation list.

Policy engine (*ZtPolicyEngine):* Serves as the central policy decision point (PDP) defined in NIST SP 800-207. It evaluates access through identity authorization (*Authorize)*, certificate checking (*AuthorizeWithCert)*, revocation enforcement, and role-based micro-segmentation (*EvaluateMicroSegmentation).* Decisions strictly default to DENY under the least privilege tenet. The engine tracks rule revisions via *GetPolicyVersion* and exposes a SHA-256 tamper-detection digest through *GetPolicyIntegrityHash.*

Context drift validator (*ContextAttributeStore, PeriodicRevalidator):* Enforces continuous re-authorization by tracking